

Name: Oraye, Louise Justine J.

Subject: 6ADET

Block Section: CS - 301

Date: September 18, 2026

Midterm Reflection Journal

Modules 4 and 5 shifted my code from terminal text to visual, interactive screens. Widgets are the building blocks of the interface, and the build method runs over and over again, returning a fresh description of the UI each time rather than editing the old one. Layout gave me trouble initially, especially using Expanded inside Row and Column. After encountering multiple “RenderFlex overflowed by X pixels” errors, I learned that parent constraints dictate available space while Expanded takes the remainder. In M4A3, using colorForType in type_style.dart taught me design consistency across screens. In M4A4's MonsterDetailScreen, I loved building the dynamic HP progress bar, though it taught me to avoid putting too much code in a single file.

Module 5 showed why state management matters. Flutter doesn't watch your variables, so simply changing a variable tells the framework nothing, setState is what marks the widget dirty so build gets called again with the new value. In M5A2, working with TextEditingController surprised me when blank entries slipped through due to extra spaces. I fixed it by checking controller.text.trim(). I also learned why ListView.builder only builds items currently on screen and recycles them while scrolling, whereas a Column builds every child immediately which, when nested in a scrollable, is where “Vertical viewport was given unbounded height” comes from. Navigation taught me that data flows downward via constructors while callbacks push info upward. M5A5 took me about 5 hours. Roughly 2 of those were building it, the other 3 were after it already worked, getting it past the autochecker. It taught me patience and that working code isn't always compliant code.

When stuck now, I check the debug console and isolate values instead of guessing. In M5A5, AI handed me an overly complex provider pattern snippet that I couldn't explain or get to work, so I threw it out and wrote a simple callback structure I actually understood. Regarding my precise confusion, when a child widget fires a callback and I call setState in the parent, I don't understand why the child sometimes still shows the old value until I navigate away and back. Leaving M5A2 to the last minute taught me the cost of delays. My habit to keep is systematic debugging, while my change is planning data flow before coding.

My final project leans heavily on Module 5 and M5A5. Looking at the Cookie Bites Order Entry Form and Order Board, core features like interactive quantity counters with live calculations and list cards directly mirror M5A5. The largest gap is data persistence. The Orders list is completely empty on every launch because nothing survives an app restart. To fix this, I chose Supabase over local storage as outlined in my project proposal so that orders persist securely across sessions. By the project's end, I want to independently produce a multi-screen app from scratch. Two weeks from now, all of my six screens will be working end to end, with an order saved to Supabase and still there after a restart.